DHA Suffa University
Department of Computer Science
Artificial Intelligence Program (CS-2001L)
Fall 2026

Lab 03 – NumPy and Pandas

Objectives

- Create and manipulate arrays with NumPy
- Understand indexing, slicing, broadcasting, and boolean masks
- Load data into a Pandas DataFrame and inspect it
- Select, filter, and sort data
- Handle missing values
- Use `groupby` and `merge` to summarise and combine data
- Explore the **Titanic dataset** step by step

Tools required: Google Colab (NumPy and Pandas are already installed)

Part 1: NumPy Arrays

NumPy is the base library for numerical work in Python. A NumPy array is faster than a list and supports maths on the whole array at once.

```
import numpy as np

a = np.array([10, 20, 30, 40, 50])
print(a)
print(type(a))          # <class 'numpy.ndarray'>
```

Creating arrays

```
print(np.zeros(4))              # [0. 0. 0. 0.]
print(np.ones((2, 3)))          # 2 rows, 3 columns of ones
print(np.arange(0, 10, 2))      # [0 2 4 6 8]
print(np.linspace(0, 1, 5))     # 5 equally spaced numbers from 0 to 1
print(np.random.randint(1, 100, 5))   # 5 random whole numbers
```

Array attributes

```
m = np.array([[1, 2, 3],
              [4, 5, 6]])

print(m.shape)      # (2, 3) -> 2 rows, 3 columns
print(m.ndim)       # 2 dimensions
```

```
print(m.size)       # 6 elements
print(m.dtype)      # int64
```

Indexing and slicing

```
a = np.array([10, 20, 30, 40, 50])
print(a[0], a[-1])      # 10 50
print(a[1:4])           # [20 30 40]

m = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])
print(m[1, 2])          # 6 (row 1, column 2)
print(m[0])             # first row: [1 2 3]
print(m[:, 1])          # second column: [2 5 8]
```

Maths on arrays and broadcasting

Operations run on every element without a loop. **Broadcasting** lets NumPy combine arrays of different shapes, for example a number with an array.

```
a = np.array([1, 2, 3, 4])
b = np.array([10, 20, 30, 40])

print(a + b)        # [11 22 33 44]
print(a * b)        # [10 40 90 160]
print(a * 2)        # [2 4 6 8]   (the number 2 is broadcast)
print(a ** 2)       # [1 4 9 16]

m = np.array([[1, 2, 3],
              [4, 5, 6]])
print(m + np.array([10, 20, 30]))   # the row is added to every row of m
```

Useful functions

```
scores = np.array([55, 78, 92, 61, 85])

print(scores.sum())         # 371
print(scores.mean())        # 74.2
print(scores.max(), scores.min())
print(scores.std())         # standard deviation
print(np.sort(scores))
print(scores.argmax())      # index of the largest value: 2
```

Boolean masks (filtering)

```
scores = np.array([55, 78, 92, 61, 85])

print(scores > 70)              # [False  True  True False  True]
print(scores[scores > 70])      # [78 92 85]
```

Reshaping

```
x = np.arange(1, 13)            # 12 numbers
print(x.reshape(3, 4))          # 3 rows, 4 columns
```

Task 1.1

1. Create an array of the numbers 1 to 20. Print only the even numbers using a boolean mask
2. Create a 3 x 3 array of numbers 1 to 9. Print its second row and its last column
3. Create an array of 6 temperatures in Celsius and convert all of them to Fahrenheit in one line
4. Print the mean, maximum, and minimum of the array `[12, 45, 7, 33, 90, 21]`

Part 2: Pandas Series and DataFrames

Pandas works with table-like data. A **DataFrame** is a table with rows and columns, like an Excel sheet. A **Series** is a single column.

```
import pandas as pd

data = {
    "name": ["Ali", "Sara", "Omar", "Hina"],
    "age": [20, 21, 19, 22],
    "marks": [78, 92, 65, 88]
}

df = pd.DataFrame(data)
print(df)
```

Reading a CSV file

```
# From a file you uploaded to Colab
df = pd.read_csv("students.csv")

# Or directly from a web address
url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv"
titanic = pd.read_csv(url)
```

Inspecting a DataFrame

Command	What it shows
`df.head()`	First 5 rows
`df.tail()`	Last 5 rows
`df.shape`	(rows, columns)
`df.columns`	Column names
`df.dtypes`	Data type of each column
`df.info()`	Columns, types, and non-null counts
`df.describe()`	Count, mean, min, max, and quartiles of numeric columns

Task 2.1

Create a DataFrame of 5 students with the columns `name`, `department`, and `cgpa`. Print its shape, its column names, and the output of `describe()`.

Part 3: Selecting, Filtering, and Sorting

```
print(df["name"])               # one column (a Series)
print(df[["name", "marks"]])    # several columns (a DataFrame)
```

loc and iloc

```
print(df.iloc[0])               # first row, by position
print(df.iloc[1:3])             # rows 1 and 2, by position
print(df.loc[0, "name"])        # row label 0, column "name"
```

Filtering rows with conditions

```
print(df[df["marks"] > 80])                          # marks above 80
print(df[(df["marks"] > 70) & (df["age"] < 22)])     # AND: use &
print(df[(df["age"] == 19) | (df["age"] == 22)])     # OR: use |
```

Use `&` and `|` (not `and` / `or`), and put each condition in brackets.

Sorting and adding columns

```
df = df.sort_values("marks", ascending=False)
df["passed"] = df["marks"] >= 70
df["percentage"] = df["marks"] / 100 * 100
print(df)
```

Task 3.1

Using your DataFrame from Task 2.1:

1. Print only the students whose CGPA is above 3.0
2. Sort the students by CGPA from highest to lowest
3. Add a new column `honors` that is `True` when the CGPA is 3.5 or more

---

Part 4: Missing Values

Real data is often incomplete. Missing values appear as `NaN`.

```
import numpy as np

df2 = pd.DataFrame({
    "name": ["Ali", "Sara", "Omar", "Hina"],
    "age": [20, np.nan, 19, 22],
    "city": ["Karachi", "Lahore", None, "Karachi"]
})
```

```
print(df2.isnull())             # True where a value is missing
print(df2.isnull().sum())       # number of missing values per column
```

Handling missing values

```
# Option 1: drop rows that have any missing value
clean = df2.dropna()

# Option 2: fill missing values
df2["age"] = df2["age"].fillna(df2["age"].mean())       # numbers: use the mean
df2["city"] = df2["city"].fillna("Unknown")             # text: use a fixed word
print(df2)
```

Rule of thumb: drop rows when very few are affected. Otherwise fill them, using the mean or median for numbers and the most common value for categories.

Task 4.1

Create a DataFrame with a few missing values in two columns. Count the missing values per column, fill the numeric column with its median, and fill the text column with the word `"Unknown"`.

Part 5: groupby and merge

groupby: summarise by category

```
sales = pd.DataFrame({
    "city": ["Karachi", "Lahore", "Karachi", "Lahore", "Karachi"],
    "amount": [100, 200, 150, 50, 300]
})

print(sales.groupby("city")["amount"].sum())
print(sales.groupby("city")["amount"].mean())
print(sales.groupby("city")["amount"].agg(["count", "sum", "mean"]))
```

merge: combine two tables using a common column

```
students = pd.DataFrame({
    "roll": [1, 2, 3],
    "name": ["Ali", "Sara", "Omar"]
})
marks = pd.DataFrame({
    "roll": [1, 2, 4],
    "marks": [80, 90, 70]
})

print(pd.merge(students, marks, on="roll"))                  # only matching rolls
print(pd.merge(students, marks, on="roll", how="left"))      # keep every student
```

Task 5.1

1. Using `sales`, find the city with the highest total amount
2. Create two small tables, `employees` (id, name) and `salaries` (id, salary), and merge them on `id`

Part 6: Lab Project – Explore the Titanic Dataset

Goal: Use Pandas to explore a real dataset and answer questions about the passengers.

Step 1: Load and inspect

```
import pandas as pd

url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv"
df = pd.read_csv(url)

print(df.shape)
print(df.head())
df.info()
print(df.describe())
```

Important columns: `survived` (1 = survived, 0 = did not), `pclass` (ticket class 1 to 3), `sex`, `age`, `fare`, `embarked`.

Step 2: Check missing values

```
print(df.isnull().sum())
```

The columns `age`, `deck`, and `embarked` have missing values.

Step 3: Clean the data

```
df["age"] = df["age"].fillna(df["age"].median())
df["embarked"] = df["embarked"].fillna(df["embarked"].mode()[0])
df = df.drop(columns=["deck"])        # too many values are missing
print(df.isnull().sum())
```

Step 4: Answer questions with the data

```
# 1. How many passengers survived?
print(df["survived"].value_counts())

# 2. What percentage survived?
print(df["survived"].mean() * 100)

# 3. Survival rate by sex
print(df.groupby("sex")["survived"].mean())

# 4. Survival rate by ticket class
print(df.groupby("pclass")["survived"].mean())

# 5. Average fare per class
print(df.groupby("pclass")["fare"].mean())

# 6. The 5 oldest passengers
print(df.sort_values("age", ascending=False).head(5)[["sex", "age", "pclass",
"survived"]])

# 7. Children (under 12) who survived
```

```
children = df[(df["age"] < 12) & (df["survived"] == 1)]
print(len(children))
```

Task 6.1 (write your own findings)

Under your code, write 4 to 5 sentences describing what the data tells you about who was more likely to survive.

Common Errors (Cheat Sheet)

Error	Cause	Fix
`ModuleNotFoundError: No module named 'pandas'`	Library not imported or not installed	Run `import pandas as pd` (install with `pip install pandas` outside Colab)
`KeyError: 'Name'`	Column name is spelled differently (names are case-sensitive)	Print `df.columns` and copy the exact name
`ValueError: The truth value of a Series is ambiguous`	Used `and` / `or` in a filter	Use `&` / `` ` ``
`ValueError: cannot reshape array`	Total number of elements does not match the new shape	Check `array.size` first
`FileNotFoundError`	CSV file was not uploaded or the name is wrong	Upload the file in Colab and check the name
Changes to `df` seem to disappear	Result was not assigned back	Write `df = df.sort_values(...)`

Lab Tasks for Student Practice

Solve the following 5 tasks on your own. Paste your code and a screenshot of the output for each task in your lab file.

Task 1: NumPy Marks Analysis (Easy)

Create a NumPy array of the marks of 10 students. Print the mean, highest, lowest, and standard deviation. Then print how many students scored above the mean.

Task 2: Normalise an Array (Easy)

Create an array `[10, 20, 30, 40, 50]`. Use broadcasting to create a new array where each value is `(value - min) / (max - min)`. Print the result.

Task 3: Student DataFrame (Medium)

Create a DataFrame of 8 students with the columns `name`, `department`, `semester`, and `cgpa`. Then:

1. Print the students of one department with CGPA above 3.0
2. Print the average CGPA of every department using `groupby`
3. Print the top 3 students by CGPA

Task 4: Clean a Messy Table (Medium)

Create a DataFrame with at least 6 rows where two columns have missing values. Count the missing values, fill the numeric column with its mean and the text column with its most common value, and print the cleaned table.

Task 5: Titanic Deep Dive (Challenging)

Using the Titanic dataset, answer these questions with code and one sentence of explanation each:

1. What is the survival rate for each combination of `sex` and `pclass`?
2. Create a new column `family_size` = `sibsp` + `parch` + 1. Is survival higher for people travelling alone?
3. Create an `age_group` column (child, teen, adult, senior) using conditions, and show the survival rate of each group
4. Which embarkation port (`embarked`) had the highest average fare?

Submission Instructions

- Paste the **code and output** of every task in your lab file
- Write **comments** in your code wherever needed
- Name your notebook: `Lab4_RollNumber.ipynb`
- Complete **Tasks 1 to 5** before the next lab

End of Lab 04. Next lab: Data Visualization, EDA and Preprocessing